HDOC Day-7 Activity

Question 1

Write a C program to find angular frequency when frequency is given.

Step 1: Understand the Problem

Problem statement: Write a C program to find angular frequency when frequency is given.

Input: Frequency f in hertz (Hz).

Output: Angular frequency ω in radians per second (rad/s).

Formula: $\omega = 2\pi f$

Step 2: Standard Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result (Pass/Fail)
1	1 Hz	6.28 rad/s	6.28 rad/s	Pass
2	2 Hz	12.57 rad/s	12.57 rad/s	Pass
3	5 Hz	31.42 rad/s	31.42 rad/s	Pass

Step 3: Algorithm and Evaluation

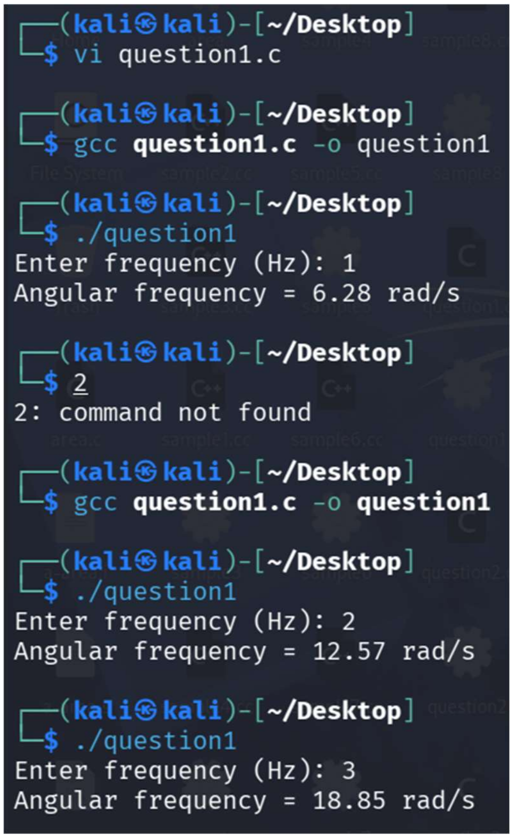
- Start.
- Read frequency f in Hz.
- Use $\pi = 3.141592653589793$.
- Calculate angular frequency: $\omega = 2 \times \pi \times f$.
- Display ω in rad/s.
- Stop.

Algorithm evaluation: Applying the algorithm to the three test cases gives the expected values shown in Step 2, so the algorithm is correct for these test cases.

Step 4: C Program

```
#include <stdio.h>

int main()
{
    double frequency, angular_frequency;
    const double PI = 3.141592653589793;

    printf("Enter frequency (Hz): ");
    scanf("%lf", &frequency);

    angular_frequency = 2 * PI * frequency;

    printf("Angular frequency = %.2f rad/s\n", angular_frequency);
    return 0;
}
```

Step 5: Compile, Execute and Evaluate



Run the program three times using the following test inputs. Compare the displayed output with the expected output in Step 2. If they match, mark the result as Pass.

Test Case 1: enter 1 → expected output: 6.28 rad/s

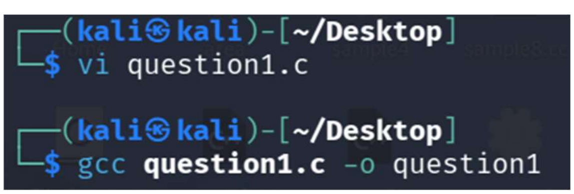
Test Case 2: enter 2 → expected output: 12.57 rad/s

Test Case 3: enter 5 → expected output: 31.42 rad/s

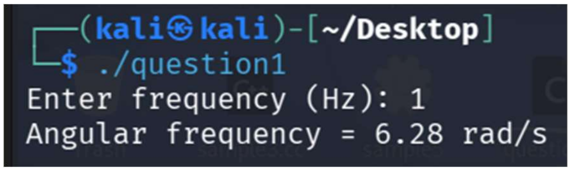
Step 6: Screenshots of Compilation and Execution

Take screenshots in Kali Linux showing the compile command and the output for each test case. Paste them in the spaces below in the Word file.

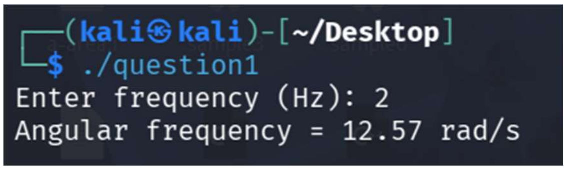
Compilation screenshot



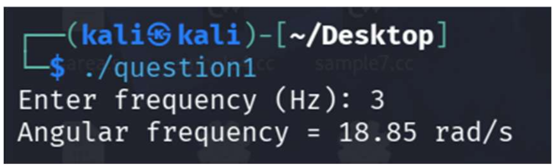
Test Case 1 execution screenshot



Test Case 2 execution screenshot



Test Case 3 execution screenshot



Question 2

Write a C program to find voltage across a capacitor when charge and capacitance are given.

Step 1: Understand the Problem

Problem statement: Write a C program to find voltage across a capacitor when charge and capacitance are given.

Input: Charge Q in coulombs (C) and capacitance C in farads (F).

Output: Voltage V across the capacitor in volts (V).

Formula: V = Q / C

Step 2: Standard Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result (Pass/Fail)
1	Q=10 C, C=2 F	5.00 V	5.00 V	Pass
2	Q=12 C, C=3 F	4.00 V	4.00 V	Pass
3	Q=25 C, C=5 F	5.00 V	5.00 V	Pass

Step 3: Algorithm and Evaluation

- Start.
- Read charge Q in coulombs.
- Read capacitance C in farads.
- Check that capacitance is not zero.
- Calculate voltage: V = Q / C.
- Display V in volts.
- Stop.

Algorithm evaluation: Applying the algorithm to the three test cases gives the expected values shown in Step 2, so the algorithm is correct for these test cases.

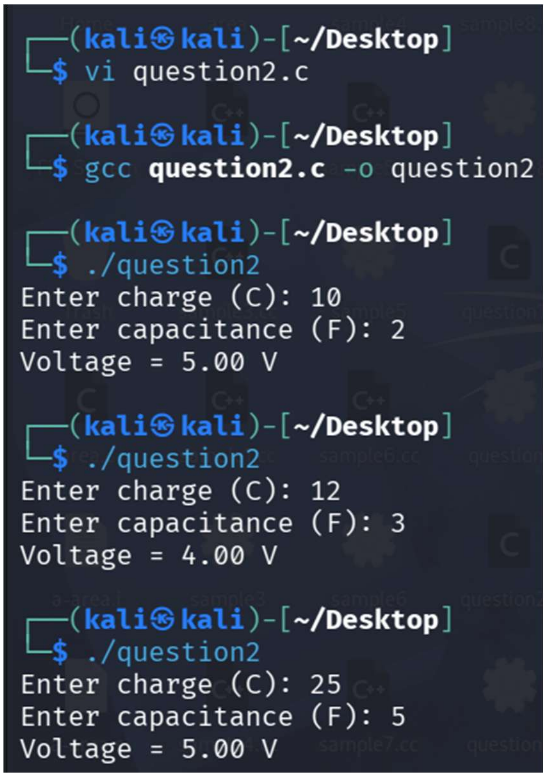
Step 4: C Program (write this in Kali Linux)

```
#include <stdio.h>

int main()
{
    double charge, capacitance, voltage;

    printf("Enter charge (C): ");
    scanf("%lf", &charge);
    printf("Enter capacitance (F): ");
    scanf("%lf", &capacitance);

    if (capacitance == 0) {
        printf("Capacitance cannot be zero.\n");
        return 1;
```

```
    }

    voltage = charge / capacitance;
    printf("Voltage = %.2f V\n", voltage);
    return 0;
}
```

Step 5: Compile, Execute and Evaluate

In Kali Linux, open Terminal and follow these commands:



Run the program three times using the following test inputs. Compare the displayed output with the expected output in Step 2. If they match, mark the result as Pass.

Test Case 1: enter charge 10, capacitance 2 → expected output: 5.00 V

Test Case 2: enter charge 12, capacitance 3 → expected output: 4.00 V

Test Case 3: enter charge 25, capacitance 5 → expected output: 5.00 V

Step 6: Screenshots of Compilation and Execution

Take screenshots in Kali Linux showing the compile command and the output for each test case. Paste them in the spaces below in the Word file.

Compilation screenshot

```
┌──(kali㉿kali)-[~/Desktop]
└─$ vi question2.c

┌──(kali㉿kali)-[~/Desktop]
└─$ gcc question2.c -o question2
```

Test Case 1 execution screenshot

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./question2
Enter charge (C): 10
Enter capacitance (F): 2
Voltage = 5.00 V
```

Test Case 2 execution screenshot

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./question2
Enter charge (C): 12
Enter capacitance (F): 3
Voltage = 4.00 V
```

Test Case 3 execution screenshot

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./question2
Enter charge (C): 25
Enter capacitance (F): 5
Voltage = 5.00 V
```

Question 3

Write a C program to find decay constant when half-life is given.

Step 1: Understand the Problem

Problem statement: Write a C program to find decay constant when half-life is given.

Input: Half-life T½ in seconds (s).

Output: Decay constant λ in per second (s^{-1}).

Formula: λ = ln(2) / T½

Step 2: Standard Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result (Pass/Fail)
1	10 s	0.069315 s^-1	0.069315 s^-1	Pass
2	20 s	0.034657 s^-1	0.034657 s^-1	Pass
3	5 s	0.138629 s^-1	0.138629 s^-1	Pass

Step 3: Algorithm and Evaluation

- Start.
- Read half-life T½ in seconds.
- Check that half-life is greater than zero.
- Calculate λ = ln(2) / T½.
- Display λ in s^-1.
- Stop.

Algorithm evaluation: Applying the algorithm to the three test cases gives the expected values shown in Step 2, so the algorithm is correct for these test cases.

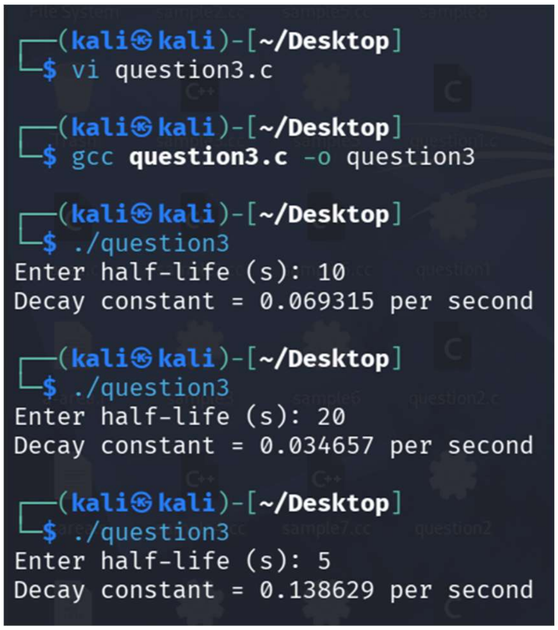
Step 4: C Program (write this in Kali Linux)

```
#include <stdio.h>
#include <math.h>

int main()
{
    double half_life, decay_constant;

    printf("Enter half-life (s): ");
    scanf("%lf", &half_life);

    if (half_life <= 0) {
        printf("Half-life must be greater than zero.\n");
        return 1;
    }

    decay_constant = log(2.0) / half_life;
    printf("Decay constant = %.6f per second\n", decay_constant);
```

```
    return 0;
}
```

Step 5: Compile, Execute and Evaluate

In Kali Linux, open Terminal and follow these commands:



Run the program three times using the following test inputs. Compare the displayed output with the expected output in Step 2. If they match, mark the result as Pass.

Test Case 1: enter 10 → expected output: 0.069315 per second

Test Case 2: enter 20 → expected output: 0.034657 per second

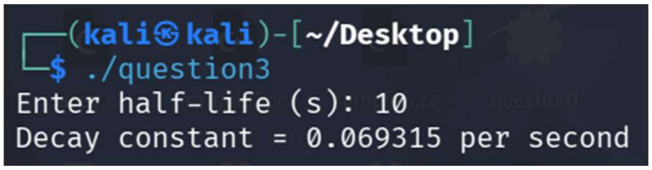
Test Case 3: enter 5 → expected output: 0.138629 per second

Step 6: Screenshots of Compilation and Execution

Take screenshots in Kali Linux showing the compile command and the output for each test case. Paste them in the spaces below in the Word file.

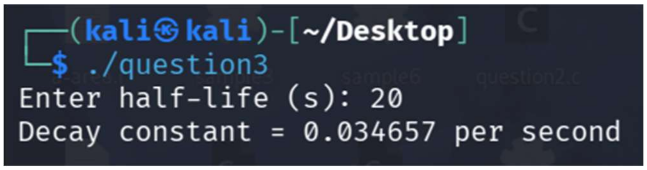
Compilation screenshot

Test Case 1 execution screenshot



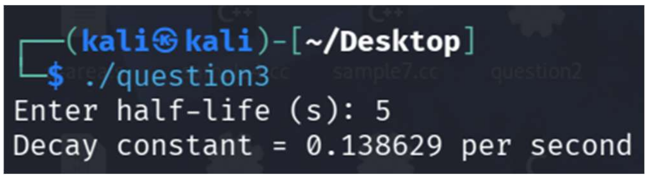
```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./question3
Enter half-life (s): 10
Decay constant = 0.069315 per second
```

Test Case 2 execution screenshot



```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./question3
Enter half-life (s): 20
Decay constant = 0.034657 per second
```

Test Case 3 execution screenshot



```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./question3
Enter half-life (s): 5
Decay constant = 0.138629 per second
```